

DBMS QUESTION

1. What are the ACID properties in a DBMS?

- **Atomicity:** Ensures that a transaction is treated as a single, indivisible unit. Either all operations in the transaction execute completely, or none at all.
- **Consistency:** Ensures that a transaction brings the database from one valid state to another, maintaining all predefined rules and constraints.
- **Isolation:** Ensures that concurrently executing transactions do not interfere with each other.
- **Durability:** Ensures that once a transaction has been committed, it will remain committed even in the case of a system failure.

2. What is Database Normalization, and what are the common forms?

- **Normalization** is the process of organizing data in a database to reduce redundancy and improve data integrity.
- **1NF (First Normal Form):** Ensures all attributes are atomic (no repeating groups or arrays).
- **2NF:** Must be in 1NF, and all non-key attributes must be fully functionally dependent on the primary key.
- **3NF:** Must be in 2NF, and there should be no transitive dependencies (non-key attributes should not depend on other non-key attributes).
- **BCNF (Boyce-Codd Normal Form):** A stricter version of 3NF where every determinant must be a candidate key.

3. What is a Self-Join, and when would you use it?

- A **Self-Join** is a regular join operation where a table is joined with itself.
- It is heavily used when dealing with hierarchical data or comparing rows within the same table. For example, in an Employees table where each employee has a manager_id that references the employee_id in the same table, a self-join is used to pair employees with their respective managers.

4. What is the difference between a Clustered and a Non-Clustered Index?

- **Clustered Index:** Dictates the physical order of the data on the disk. Because the data rows can only be sorted in one way, a table can only have exactly one clustered index (usually the Primary Key).
- **Non-Clustered Index:** Creates a separate logical structure (like a lookup table) that points to the physical rows. A table can have multiple non-clustered indexes, which makes searching faster but requires extra storage space.

5. Explain the CAP Theorem.

- The CAP Theorem states that a distributed database system can only guarantee two out of the following three characteristics simultaneously:
 - **Consistency:** Every read receives the most recent write or an error.
 - **Availability:** Every request receives a (non-error) response, without the guarantee that it contains the most recent write.
 - **Partition Tolerance:** The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.

6. What is Database Sharding?

- **Sharding** is a type of horizontal partitioning where a large database is divided into smaller, faster, and more easily managed parts called "shards."
- Each shard is held on a separate database server instance, which spreads the load. It is highly critical in system design for tech giants like Google to scale databases out horizontally.

7. What is the difference between a View and a Materialized View?

- **View:** A virtual table based on the result-set of an SQL statement. It does not store data physically; the query runs dynamically every time the view is accessed.
- **Materialized View:** A database object that physically stores the results of a query on disk (highly popular in PostgreSQL). It provides much faster read performance for complex queries, but the data must be periodically refreshed.

8. What are the different Transaction Isolation Levels?

- **Read Uncommitted:** The lowest level; transactions can see uncommitted changes from other transactions (leads to Dirty Reads).
- **Read Committed:** Guarantees that any data read is committed at the moment it is read.
- **Repeatable Read:** Ensures that if a row is read twice in the same transaction, the values will be the same (prevents Non-Repeatable Reads).
- **Serializable:** The highest level; transactions are executed in such a way that the outcome is the same as if they were executed sequentially.

9. What is the difference between a Primary Key and a Unique Key?

- **Primary Key:** Uniquely identifies a record in a table. It cannot accept NULL values, and there can only be one primary key per table.
- **Unique Key:** Ensures all values in a column are distinct. However, unlike a primary key, it can accept exactly one NULL value, and a table can have multiple unique keys.

10. How do B-Trees and B+ Trees differ in database indexing?

- **B-Tree:** Both data pointers and keys are stored in internal (non-leaf) nodes as well as leaf nodes.
- **B+ Tree:** Data pointers are stored *only* at the leaf nodes, while the internal nodes only store keys for routing. Additionally, the leaf nodes are linked together like a linked list, making sequential access and range queries (like BETWEEN operations) significantly faster.